

Defense Speech — Clausify / CUAD

17 slides · ~18 minutes at a normal speaking pace · numbers in bold are the ones to land clearly

Delivery notes: the whole talk has one argument — *component scores do not describe the assembled system*. Everything else is evidence for it. If you run short on time, cut slides 3 and 4 (method detail) and keep 5, 8, 11 and 13. Never cut slide 11.

Slide 1 — Cover · 40 seconds

Good morning. I'm Afnan, and with Jerin and Shoyeb I'll be presenting our thesis on automated contract review.

The short version is this. We built a three-stage system that reads a commercial contract, finds the clauses that matter, and flags how risky each one is. It works — but most of what we found is that it works considerably less well than its individual test scores suggest. That gap is what this talk is about, and we think it is the more useful contribution.

Slide 2 — The problem, and the data · 75 seconds

Contracts are long and the important parts are short. In our dataset the median contract is **33,000 characters** — about five thousand words — and the median clause that actually matters is **196 characters**. So the document is about **sixteen times** longer than what a standard transformer can read in one pass, while the target inside it is two orders of magnitude smaller.

We work on CUAD, the Contract Understanding Atticus Dataset: **510 real commercial contracts**, with lawyers having marked every clause belonging to **41 categories**.

Three things about this data shaped everything that followed. It is split **80/20 at the contract level** — whole contracts on one side or the other, so no clause from a training contract can leak into the test set. It is severely imbalanced: some categories appear in every contract, five of them appear in **seven or fewer** of our test contracts. And it cannot be grown — the annotation reportedly cost around two million dollars of attorney time.

Slide 3 — The pipeline · 90 seconds

Here is the system. Four stages: find which categories are present, extract the clause text, attach a risk level, attach a plain-English sentence.

To handle the length problem we slide a **2,000-character window** across the contract in 1,500-character steps, score every window, and take the **maximum** as the document-level answer. That is the classical multiple-instance learning rule: the document is a bag of windows, and if any window contains the clause, the document does.

The 500-character overlap matters — it is larger than the median clause, so no clause can fall between two windows and vanish.

Before building this we measured something. We asked: if we used retrieval instead — picking the few most promising windows — what is the best recall we could possibly reach? The answer was **64 percent**, which is below what our simple baseline already achieves. So retrieval was ruled out by a measurement that took under a minute, rather than by a training run that would have taken hours.

The numbers under each stage are what I will come back to: the percentage of clauses lost at that boundary.

Slide 4 — Training and testing · 60 seconds

Everything trained on one laptop — an M4 with 16 gigabytes, no CUDA anywhere. Presence took about **49 minutes**, span **23**, the text generator **8**.

On the testing side, two things I want to be explicit about. First, **no hyperparameter was ever tuned**. Every learning rate and batch size is a published default. That protects the held-out evaluation completely — but it also means the transformer competes untuned, and I will come back to that.

Second, this table shows exactly what was held out during training for each model. The last row is a mistake we are reporting rather than hiding: our exploratory run of the summarizer monitored its loss on 300 clauses from the **test** split. It selected no checkpoint and the deployed model was rebuilt without it, so no reported number depends on it — but we should not have done it.

Slide 5 — The first result · 75 seconds

Here is the headline comparison. The full-document TF-IDF baseline scores **0.775**. Our windowed transformer scores **0.694**. The bag-of-words model wins, by eight points.

That was not what we expected, so most of this talk is about whether we should believe it.

The conjunctive ensemble — where both models must agree — reaches **0.776**, nominally first. I will show you in two slides why we do not claim that as a win.

At the bottom left you can see where the model is strong: Document Name, Parties, Governing Law are essentially solved. But five categories score **zero**, and that is why our macro-F1 is **0.572** against a micro-F1 of 0.776.

Slide 6 — Testing the alternatives · 90 seconds

Before accepting that the transformer simply loses, we tested every excuse available to it.

Was it undertrained? It saw 24,000 of 53,662 available windows. We retrained on all of them. The score moved by **minus 0.0004**. No effect.

Was it an unlucky seed? Three training runs. The gap to the baseline is **9.5 standard deviations** wide. No effect.

Was max-pooling the wrong aggregation? We tried six rules on identical window scores. The best of them, at its own best threshold, reaches 0.750 — still below the baseline. No effect.

Was the threshold wrong? This one is a real caveat, and we report it as such. The 0.5 threshold was never chosen — it is just the softmax default. Swept, the transformer reaches **0.750** instead of 0.694. So part of the gap is an operating point nobody picked.

Now look at the chart. The ensemble’s winning margin is **0.0014**. The variation from reseeding is 0.0022. From redrawing the split, 0.0063. The bootstrap interval is 0.0078. **Every error bar we can measure is larger than the result.** So we withdrew that claim.

Slide 7 — Threshold and calibration · 70 seconds

Two things we only measured after the fact, and both mattered.

First, as I said, 0.5 is a poor operating point. Second — and this one affects users directly — the score is **not calibrated**. Expected Calibration Error is **0.201**. When the model says above 0.9, the clause is actually there about **73 percent** of the time. When it says 0.5 to 0.6, it is there under **20 percent** of the time. Our application displays this number as a confidence bar, which means the bar is misleading.

What we would do about it is concrete, and it is on the slide: carve a validation split out of the 408 training contracts, choose the threshold there, and fit a Platt scaling map on the same split. Neither step costs compute, and neither touches the test set. We could not do it here because we never carved that split — which is a protocol gap, not an oversight in one experiment.

Slide 8 — When the metric disagrees with the purpose · 90 seconds

This is the result I would most like you to take away.

Our whole premise is that some clauses matter more than others — that is what the risk layer is for. So we asked: where do the errors fall?

The ensemble, our headline configuration, misses **37.5 percent of High-risk clauses**. The transformer alone — the model we have spent two slides establishing is weaker — misses **15.9 percent**. And the OR rule, which finishes *last* on aggregate micro-F1, misses only **13.6 percent**.

In absolute terms: 176 High-risk clauses in the test set. The ensemble surfaces 110 and loses 66. OR surfaces 152 and loses 24. Choosing on micro-F1 costs a signer **42 high-risk clauses** — uncapped liabilities, IP assignments, non-competes.

The reason is that two thirds of micro-F1’s mass is Low-risk categories: dates, party names, governing law. **The metric is dominated by the clauses that matter least.**

We are reporting this as a diagnosis, not switching configurations — picking OR after seeing these test results would be exactly the test-set selection we have avoided everywhere else.

Slide 9 — The span model · 70 seconds

Stage two has two separate problems.

At training time, every example was re-centred so the answer begins about 150 characters into the window. At inference the answer can be anywhere — or not in the window at all.

31.7 percent of the time, the window our presence model selects does not contain the clause. That is because max-pooling was trained to answer *does this window contain a clause*, not *which window is it in* — we reused a detector as a locator.

And even when the window is right, extraction succeeds only **39.7 percent** of the time, against 82.7 percent in the isolated benchmark. Token-F1 falls from **0.763 to 0.386**.

The model also has no way to say “not here” — so when handed the wrong window, it answers confidently anyway.

Slide 10 — The summarizer · 80 seconds

This one is the most interesting failure.

CUAD has no plain-English rewrites, so we wrote the targets ourselves: **41 sentences, one per category**. And that single decision is what broke the component.

The model learned to recognise the category and print its sentence. On the test set it emits one of those 41 templates **verbatim, 100 percent of the time** — the correct one 70 percent, a different category’s template the other 30.

Against our own templates it scores ROUGE-L **0.775**, which looks excellent. Against clause-specific references it scores **0.116**. And the bare template, with no model at all, scores **0.115** — so the model adds nothing measurable over a lookup table.

Worth noting: a template-emitting model cannot hallucinate. But when it picks the wrong template, the output is fluent, confident, and about a different provision entirely — and that is undetectable from the output alone.

Slide 11 — End to end · 90 seconds

This is the most important number in the thesis.

Everything so far was measured with the other stages held out of the way. So we ran the pipeline end to end and asked, of every gold clause: does it survive all three stages?

We start with **1,348** clauses. After presence detection, **1,052** — we lose 296, or 22 percent. After span extraction, **293**. We lose another 759.

21.7 percent survive, with a confidence interval of 19.6 to 23.9.

Multiplying the component scores predicts about **64 percent**. The pipeline delivers a third of that.

(pause here)

And the losses are not in the components — they are at the interfaces between them. Both causes are fixable without a bigger model.

Slide 12 — What we found · 60 seconds

Pulling it together. On the left, four findings that survived every check we could run. On the right, five things we believed at the start that did not survive.

We were wrong that the ensemble beat the baseline. Wrong that the summarizer summarized. Wrong that component scores describe the system. Wrong that a Longformer would not fit in memory — it does, at batch size one. And wrong that max-pooling was the ceiling — it turned out to be a secondary factor.

Five of our own starting assumptions were wrong, and we found that out by testing them.

Slides 13 and 14 — What is missing · 75 seconds

Two slides of honest gaps.

The first is what we discovered during the research. CUAD has **no plain-English targets at all** — so we wrote them, and that is precisely what collapsed the summarizer. The rare tail is effectively few-shot, with 11 to 42 training examples. Some of what we count as errors are arguably annotation disagreements. And we never carved a validation split, which is why the threshold and the calibration cannot be fixed without redrawing the protocol.

The second is what the work still lacks. Risk is assigned per **category**, not per clause — it cannot tell a ten-thousand-dollar liability cap from a ten-million-dollar one. Everything comes from 510 US contracts from SEC filings: no Bangladeshi contract, no consumer or employment agreement, no other language. And nobody reports end-to-end survival, so we had no published figure to compare our 21.7 percent against.

Slide 15 — Clausify · 55 seconds

The system is deployed as a local web application. A median contract takes **27 seconds**, worst case 32.

Three caveats it has to carry. The confidence bar is not a probability. Long contracts are truncated — we read the first 45,500 characters, and **38 percent** of CUAD contracts are longer than that. And privacy holds only because it runs locally: nothing is written to disk or logged.

The practical instruction for a user is that the risk badge and category are far more reliable than the quoted sentence underneath them. A card should be read as “look here”, not “this is the clause”.

Slides 16 and 17 — Future work · 70 seconds

Ranked by value per hour.

The first four need no GPU at all. Carve that validation split. Put the application in front of five to ten real users and watch specifically for over-trust. Select on a risk-weighted criterion rather than micro-F1. Calibrate the confidence score.

The modelling work targets the specific failures I showed you: give the span model a way to abstain, train it on the windows it will actually see, build a locator trained for locating, and grow the generation targets so the task stops being 41-way classification.

Closing · 30 seconds

To close where I started. We built a working prototype, and we spent most of our effort finding out where it fails.

If there is one transferable lesson, it is this: **a margin should be compared against its own noise before it is reported, and a pipeline should be evaluated against the harm it is supposed to prevent** — not against an average that treats a governing-law clause and an uncapped indemnity as equally important.

Thank you. We are happy to take questions.

Anticipated questions

“If the ensemble’s lead is just noise, why keep it in the pipeline?” Removing it changes nothing measurably, which is itself a finding. We keep it for the balanced error profile and because it is four times less sensitive to the random seed than the transformer inside it.

“Why didn’t you validate the risk taxonomy before building the system?” Honestly, because we did not know it would become the system’s failure mode until it already had. The risk-weighted evaluation that revealed the problem was one of the last experiments we ran, and it is itself one of our findings.

“21.7 percent sounds like the system does not work.” It means the system is a prototype, and we would rather report the number than the flattering one. It also tells us exactly where to spend the next effort — both causes are at stage boundaries, and neither needs a larger model.

“Isn’t TF-IDF beating a transformer just a sign your transformer was badly trained?” That is the first thing we checked. Full data, three seeds, six aggregation rules — none of them closes the gap. The threshold explains part of it, and we say so.

“Why is the summarizer in the thesis at all if it does not work?” Because the way it fails is a result. A 41-sentence reference set turned a generation task into classification, and ROUGE-L reported 0.775 without registering the change. That is a warning about evaluation, not just about our model.

“What would you do differently?” Carve a validation split on day one. Everything downstream — the threshold, the calibration, selecting the ensemble rule on the objective we actually care about — was blocked by not having one.